

1 Norm Expressions for Quadratic Forms

(a) First, if

$$f(x) = \|Fx\|_2^2$$

then

$$x^T Ax = \|Fx\|_2^2 \geq 0$$

for all x , so $A \succeq 0$.

Conversely, suppose $A \succeq 0$. Since A is symmetric, by the spectral theorem,

$$A = Q\Lambda Q^T$$

where all eigenvalues are nonnegative. Let $r = \text{rank}(A)$, and write

$$A = Q_r \Lambda_r Q_r^T$$

where Λ_r contains only the positive eigenvalues. Define

$$F = \Lambda_r^{1/2} Q_r^T$$

Then

$$F^T F = Q_r \Lambda_r Q_r^T = A$$

and therefore

$$x^T Ax = x^T F^T F x = \|Fx\|_2^2$$

Finally, for any such F ,

$$\text{rank}(A) = \text{rank}(F^T F) = \text{rank}(F) \leq k$$

Our construction has $k = r$, so the smallest possible size is

$$\boxed{k_{\min} = \text{rank}(A)}$$

(b) Since A is symmetric, by the spectral theorem

$$A = Q\Lambda Q^T$$

Separate the positive and negative eigenvalues:

$$A = Q_+ \Lambda_+ Q_+^T - Q_- \Lambda_- Q_-^T$$

where Λ_+ contains the positive eigenvalues of A , and Λ_- contains the absolute values of the negative eigenvalues.

Define

$$F = \Lambda_+^{1/2} Q_+^T, \quad G = \Lambda_-^{1/2} Q_-^T$$

Then

$$F^T F = Q_+ \Lambda_+ Q_+^T, \quad G^T G = Q_- \Lambda_- Q_-^T$$

so

$$x^T Ax = x^T (F^T F - G^T G) x = \|Fx\|_2^2 - \|Gx\|_2^2$$

The smallest possible numbers of rows are

$$\boxed{k_F = n_+(A), \quad k_G = n_-(A)}$$

where $n_+(A)$ and $n_-(A)$ are the numbers of positive and negative eigenvalues of A , counting multiplicity. Zero eigenvalues require no rows in either F or G .

2 Properties of Symmetric Matrices

(a) **True.** If $P \geq 0$, then

$$(P + Q) - Q = P \geq 0$$

Therefore

$$P + Q \geq Q$$

(b) **True.** If $P \geq Q$, then simply multiply both sides by -1

$$-P \leq -Q$$

(c) **True.** Let $P > 0$, and let $x \neq 0$. Since P is invertible, $y = P^{-1}x$ is also nonzero. Then

$$x^T P^{-1} x = (Py)^T P^{-1} (Py) = y^T P y > 0$$

so

$$P^{-1} > 0$$

(d) **True.** Assume

$$P \geq Q > 0$$

Define

$$Z = Q^{-1/2} P Q^{-1/2}$$

Then

$$Z - I = Q^{-1/2} (P - Q) Q^{-1/2} \geq 0$$

so

$$Z \geq I$$

The eigenvalues of Z are at least one, so the eigenvalues of Z^{-1} are at most one. Therefore

$$Z^{-1} \leq I$$

Now

$$P^{-1} = Q^{-1/2} Z^{-1} Q^{-1/2}, \quad Q^{-1} = Q^{-1/2} I Q^{-1/2}$$

and hence

$$Q^{-1} - P^{-1} = Q^{-1/2} (I - Z^{-1}) Q^{-1/2} \geq 0$$

so

$$P^{-1} \leq Q^{-1}$$

(e) **False.** Take

$$P = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad Q = \begin{bmatrix} -2 & 0 \\ 0 & 0 \end{bmatrix}$$

Then

$$P - Q = \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix} \geq 0$$

so $P \geq Q$. But

$$P^2 - Q^2 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} - \begin{bmatrix} 4 & 0 \\ 0 & 0 \end{bmatrix} = \begin{bmatrix} -3 & 0 \\ 0 & 1 \end{bmatrix}$$

which is not positive semidefinite. Therefore

$$P^2 \not\geq Q^2$$

3 Squareroot and Logarithm of a Diagonalizable Matrix

(a) Let

$$D = \text{diag}(\mu_1, \dots, \mu_n), \quad \mu_i^2 = \lambda_i$$

and define

$$B = TDT^{-1}$$

Then

$$B^2 = (TDT^{-1})(TDT^{-1}) = TD^2T^{-1}$$

Since

$$D^2 = \text{diag}(\mu_1^2, \dots, \mu_n^2) = \text{diag}(\lambda_1, \dots, \lambda_n) = \Lambda$$

we obtain

$$\boxed{B^2 = T\Lambda T^{-1} = A}$$

Thus B is a square root of A .

In general this construction may give a complex square root. When the eigenvalues and square-root choices are paired so that B is real, it gives a real square root. In particular, if all λ_i are real and nonnegative, we can take

$$\mu_i = \sqrt{\lambda_i}$$

which gives the conventional square root $A^{1/2}$.

(b) Since A is invertible, $\lambda_i \neq 0$. Choose ν_i such that

$$e^{\nu_i} = \lambda_i$$

and define

$$B = T \text{diag}(\nu_1, \dots, \nu_n) T^{-1}$$

Then

$$e^B = T \text{diag}(e^{\nu_1}, \dots, e^{\nu_n}) T^{-1} = T\Lambda T^{-1} = A$$

Therefore,

$$\boxed{\log A = T \text{diag}(\nu_1, \dots, \nu_n) T^{-1}}$$

The logarithm is not unique, since each nonzero λ_i has complex logarithms differing by multiples of $2\pi i$.

If B is required to be real, a real logarithm does not always exist. For a real diagonalizable A , it exists iff each negative real eigenvalue has even multiplicity. In particular, if all eigenvalues are positive, we can choose

$$\nu_i = \ln \lambda_i$$

which gives a real logarithm.

4 Ellipsoids

(a) For a 2×2 symmetric positive definite matrix A , use the eigendecomposition

$$A = Q\Lambda Q^T, \quad \Lambda = \text{diag}(\lambda_1, \lambda_2), \quad \lambda_i > 0$$

The boundary of the ellipse

$$E = \{x \in \mathbb{R}^2 \mid x^T A x = 1\}$$

can be parametrized by

$$x(\theta) = Q \begin{bmatrix} 1/\sqrt{\lambda_1} & 0 \\ 0 & 1/\sqrt{\lambda_2} \end{bmatrix} \begin{bmatrix} \cos \theta \\ \sin \theta \end{bmatrix}, \quad 0 \leq \theta \leq 2\pi$$

The code at the end.

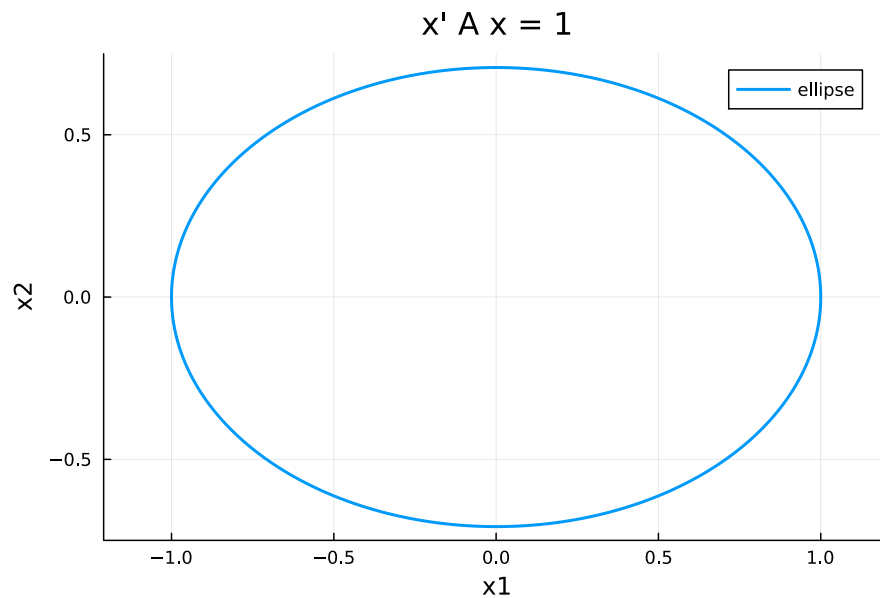
(b) For

$$A = \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix}$$

the ellipse equation is

$$x_1^2 + 2x_2^2 = 1$$

so the semiaxis lengths are 1 in the x_1 direction and $1/\sqrt{2}$ in the x_2 direction.



(c) For

$$A = \begin{bmatrix} 0.2 & -0.1 \\ -0.1 & 0.4 \end{bmatrix}$$

the eigenvalues are

$$\lambda_1 = 0.158579, \quad \lambda_2 = 0.441421$$

so the semiaxis lengths are

$$\frac{1}{\sqrt{\lambda_1}} = 2.511179, \quad \frac{1}{\sqrt{\lambda_2}} = 1.505128$$

The corresponding semiaxis directions are approximately

$$\begin{bmatrix} -0.923880 \\ -0.382683 \end{bmatrix}, \quad \begin{bmatrix} -0.382683 \\ 0.923880 \end{bmatrix}$$

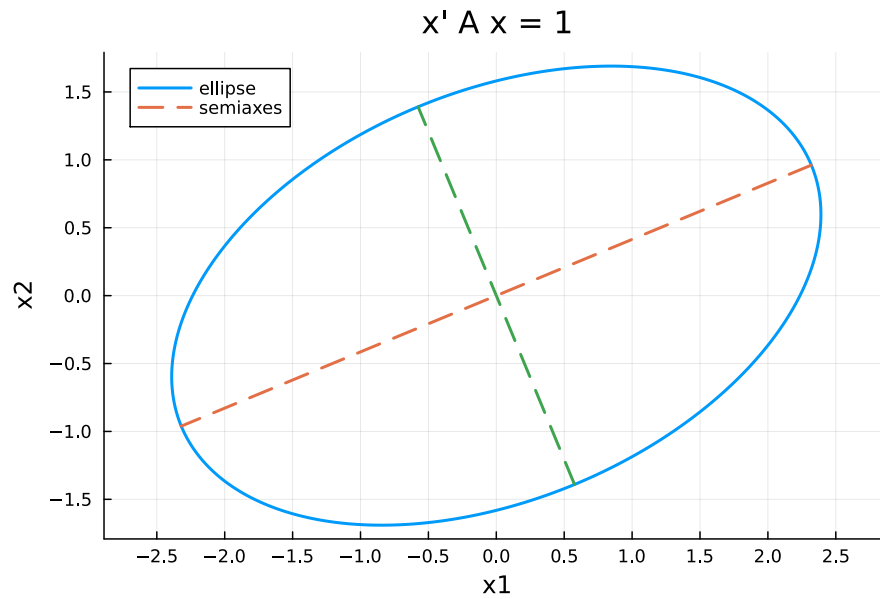


Figure 1: Ellipse with semi-axes shown

(d) Put the sensor vectors into

$$B = \begin{bmatrix} b_1 & b_2 & b_3 \end{bmatrix}$$

Then

$$y = B^T x$$

and the set $\|y\|_2 \leq 1$ is

$$\|B^T x\|_2^2 \leq 1$$

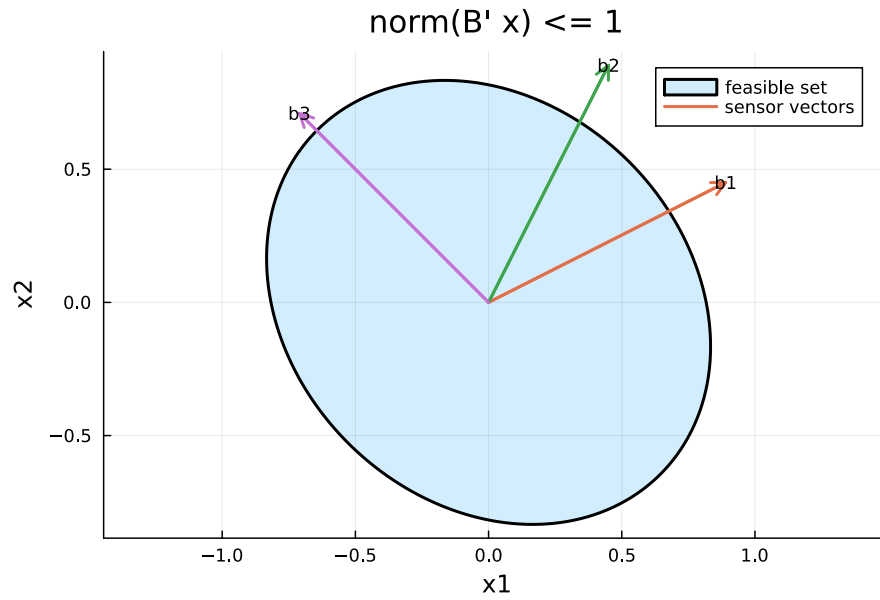
Equivalently,

$$x^T B B^T x \leq 1$$

For the given vectors,

$$B B^T = \begin{bmatrix} 1.4987 & 0.2969 \\ 0.2969 & 1.4987 \end{bmatrix}$$

so this set is an ellipse centered at the origin. The feasible set consists of the interior of the blue ellipse together with its boundary.



4.1 Code

```

1 using LinearAlgebra
2 using Plots
3
4 function ellipse_points(A; npoints=500)
5     E = eigen(Symmetric(A))
6     theta = range(0, 2pi, length=npoints)
7     circle = hcat(cos.(theta), sin.(theta))'
8     points = E.vectors * Diagonal(1 ./ sqrt.(E.values)) * circle
9     return points[1, :], points[2, :], E.values, E.vectors
10 end
11
12 function tick_locations(values; step=0.5)
13     lower = step * floor(minimum(values) / step)
14     upper = step * ceil(maximum(values) / step)
15     return lower:step:upper
16 end
17
18 function plot_ellipse(A; show_semiaxes=false, vectors=nothing,
19     fill_feasible=false, tick_step=0.5)
20     x, y, vals, vecs = ellipse_points(A)
21     p = plot(x, y;
22         seriestype=fill_feasible ? :shape : :path,
23         aspect_ratio=:equal,
24         xticks=tick_locations(x; step=tick_step),
25         yticks=tick_locations(y; step=tick_step),
26         fillalpha=fill_feasible ? 0.18 : 0.0,
27         label=fill_feasible ? "feasible set" : "ellipse")
28
29     if show_semiaxes
30         for i in 1:2
31             axis = vecs[:, i] / sqrt(vals[i])
32             plot!(p, [-axis[1], axis[1]], [-axis[2], axis[2]]);

```

```
33         linestyle=:dash, label=i == 1 ? "semiaxes" : false)
34     end
35 end
36
37 if vectors != nothing
38     for i in 1:size(vectors, 2)
39         b = vectors[:, i]
40         plot!(p, [0.0, b[1]], [0.0, b[2]]);
41         arrow=true, label=i == 1 ? "sensor vectors" : false)
42     end
43 end
44
45 return p
46 end
47
48 A_b = [1.0 0.0;
49        0.0 2.0]
50
51 A_c = [0.2 -0.1;
52        -0.1 0.4]
53
54 b1 = [0.89, 0.45]
55 b2 = [0.45, 0.89]
56 b3 = [-0.71, 0.71]
57 B = hcat(b1, b2, b3)
58 A_d = B * B'
59
60 plot_ellipse(A_b)
61 plot_ellipse(A_c; show_semiaxes=true)
62 plot_ellipse(A_d; vectors=B, fill_feasible=true)
```

5 Recovering an Ellipsoid from Boundary Points

For $n = 4$, write the symmetric matrix A as

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{12} & a_{22} & a_{23} & a_{24} \\ a_{13} & a_{23} & a_{33} & a_{34} \\ a_{14} & a_{24} & a_{34} & a_{44} \end{bmatrix}$$

For a sample $x = (x_1, x_2, x_3, x_4)$, the equation $x^T A x = 1$ becomes

$$a_{11}x_1^2 + 2a_{12}x_1x_2 + 2a_{13}x_1x_3 + 2a_{14}x_1x_4 + a_{22}x_2^2 + 2a_{23}x_2x_3 + 2a_{24}x_2x_4 + a_{33}x_3^2 + 2a_{34}x_3x_4 + a_{44}x_4^2 = 1$$

This is linear in the unknown entries of A . Define

$$\theta = [a_{11} \ a_{12} \ a_{13} \ a_{14} \ a_{22} \ a_{23} \ a_{24} \ a_{33} \ a_{34} \ a_{44}]^T$$

and form a row $\phi(x)^T$ from the coefficients in the equation above. For all boundary samples,

$$\Phi\theta \approx \mathbf{1}$$

We can estimate θ by least squares:

$$\hat{\theta} = \arg \min_{\theta} \|\Phi\theta - \mathbf{1}\|_2^2$$

Then we can reconstruct $\hat{A}$ from the entries in $\hat{\theta}$.

```
n = 4, N = 100
A =
4x4 Matrix{Float64}:
 1.43495  0.050855 -2.8708  1.61196
 0.050855 0.414349 -1.38418 0.898785
-2.8708 -1.38418 16.1368 -11.8447
 1.61196 0.898785 -11.8447  9.31723
eigenvalues(A) =
4-element Vector{Float64}:
 0.046747
 0.421146
 1.233411
25.602027
```

All eigenvalues are positive, so $\hat{A} > 0$. Therefore

$$\{x \in \mathbb{R}^4 \mid x^T \hat{A} x = 1\}$$

is a genuine bounded ellipsoid.

To check consistency with the data, I computed

$$r_k = (x^{(k)})^T \hat{A} x^{(k)}$$

For exact boundary points, all r_k would equal one. In the data,

```
min x'Ax = 0.697006
max x'Ax = 1.177020
mean x'Ax = 0.993005
RMS boundary error = 0.083636
max abs boundary error = 0.302994
```


The RMS boundary error and maximum absolute deviation are

$$\frac{\|r - \mathbf{1}\|_2}{\sqrt{N}} = 0.083636, \quad \max_k |r_k - 1| = 0.302994$$

Thus, a typical boundary value differs from 1 by about 8.4%, and the largest deviation is approximately 0.303. These measures are more informative than the mean because positive and negative errors cannot cancel.

The mean value is

$$\frac{1}{N} \sum_{k=1}^N r_k = 0.993005$$

The mean is close to 1, so the fit has little overall bias. Together with the positive eigenvalues of $\hat{A}$, these errors indicate a reasonable fit to the noisy boundary data.

5.1 Code

```

1 using LinearAlgebra
2 using Statistics
3
4 include(joinpath(@__DIR__, "readclassjson.jl"))
5
6 function symmetric_design(X)
7     n, N = size(X)
8     pairs = Tuple{Int, Int}[]
9     Phi = zeros(N, div(n * (n + 1), 2))
10
11     col = 1
12     for i in 1:n, j in i:n
13         push!(pairs, (i, j))
14         Phi[:, col] = i == j ? X[i, :] .^ 2 :
15                               2 .* X[i, :] .* X[j, :]
16         col += 1
17     end
18
19     return Phi, pairs
20 end
21
22 function symmetric_matrix(theta, pairs, n)
23     A = zeros(n, n)
24     for (col, (i, j)) in enumerate(pairs)
25         A[i, j] = theta[col]
26         A[j, i] = theta[col]
27     end
28     return Symmetric(A)
29 end
30
31 data = readclassjson(joinpath(@__DIR__, "ellip_bdry_data.json"))
32 X = data["X"]
33 n, N = size(X)
34
35 Phi, pairs = symmetric_design(X)
36 theta = Phi \ ones(N)

```

```
37 A = symmetric_matrix(theta, pairs, n)
38
39 values = [dot(X[:, k], A * X[:, k]) for k in 1:N]
40 errors = values .- 1
41 evals = eigvals(A)
```